

TEMARL

Transformer-Enhanced Multi-Agent Reinforcement Learning for Adaptive Honeypot Deception

Progress update: genuine multi-agent learning, results, and comparison plan

Apurba Roy · Tushit Roy · Mark Dipto Gomes · Farjana Akter Shumi

Supervisor: Md. Sakir Hossain

BRAC University · September 2026

At a glance

What TEMARL does	Four defender agents, one per network zone, deploy MITRE D3FEND decoys. A language model reads the attacker's technique history and tells every agent what the attacker is likely to do next.
What we built	A new, genuinely multi-agent environment in which the agents must coordinate, trained with multi-agent RL (MAPPO) and validated before any result was trusted.
Main result	On attacks seen in training, the team keeps attackers engaged for 8.4 steps, against 4.4 for the best fixed decoys. Attacker-intent modelling adds 28%. Transformer, GRU and LSTM perform the same.
Honest limit	On a completely unseen attack script the team does not generalise: it falls to 3.6 steps and loses to a simple reactive rule.
Next step	Compare with published approaches through adapted baselines inside the same environment.

1. The language models we implemented

Each model reads the attacker's last 16 MITRE ATT&CK techniques and turns them into a 64-number "intent vector" shared with all four defender agents.

Model	How it reads the attack history	Parameters	Why it is included
Transformer (RoPE)	Attention: looks at all past techniques at once; rotary position encoding keeps their order	67,072	The thesis model
GRU	Recurrent: reads techniques one at a time and keeps a running memory	67,168	Standard recurrent comparison
LSTM	Recurrent, with a separate long-term memory cell	67,152	Second recurrent comparison
SetEncoder	Treats the history as an unordered bag of techniques	67,272	Tests whether order matters
NoHistory	No model: the intent vector is always zero	0	Tests whether intent matters at all

Kept fair on purpose

- **Same size:** all four models are matched to within 0.3% of the Transformer's parameter count.
- **Same training:** each is first trained to predict the attacker's next technique, then frozen before the agents learn.
- **Prediction accuracy on CAM-LDS attacks (Top-1):** Transformer 45%, GRU 45%, LSTM 45%, SetEncoder 33%. Always guessing the most common technique scores 6%; a "last technique → next technique" lookup table scores 37%.

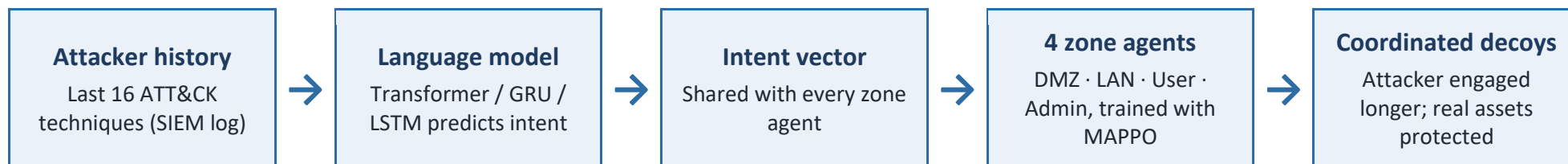
2. The new multi-agent environment

2.1 Why we had to build it

An audit of the previous system (v5) showed it did not match what the proposal promised.

Problem found in the previous system	Why it mattered
Not reinforcement learning. The agents copied a scripted "correct answer" (behaviour cloning), because RL (PPO) never beat a single fixed decoy.	The thesis title promises multi-agent reinforcement learning.
Nothing to coordinate. Reward depended only on the one agent watching the attacker's zone.	Four separate single-agent problems, not a team problem.
Identical agents. All four agents were trained to give the same answer.	No team behaviour could emerge.
Untrained placement. The part of the network that chooses where to place a decoy was never trained.	Weakened results on larger networks.
Main metric ran backwards. Catching the attacker ended the episode, so a better defence produced a shorter dwell time.	A defender doing nothing scored the highest dwell time.

2.2 How we built it



1. **Engagement, not capture.** A deceived attacker keeps interacting with decoys; every interaction step counts as dwell time, so a better defence now scores higher.
2. **Three rules that force coordination.** Shared capacity: at most two decoys run at full quality. Breadcrumbs: a fake credential or file only works if another agent has a decoy waiting. Shared exposure: one unrealistic decoy reveals the deception to the attacker for the whole team.
3. **Real attacker behaviour.** The attacker follows the seven CAM-LDS attack scripts.
4. **Validated before training.** Five checks, fixed in advance, all passed. Coordination alone is worth 33% of the best achievable score.
5. **Trained with MAPPO,** after a learning check confirmed the team really learns (it captured 78.5% of the achievable gain).
6. **Pre-registered.** Every hypothesis and statistical test was written down and committed before the final experiment ran.

2.3 The papers behind each design choice

Design element	Paper	Link
Learning algorithm (MAPPO)	Yu et al., "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games", NeurIPS 2022	arxiv.org/abs/2103.01955
Comparison learner (IPPO)	de Witt et al., "Is Independent Learning All You Need in the StarCraft Multi-Agent Challenge?", 2020	arxiv.org/abs/2011.09533
Problem structure (Dec-POMDP)	Oliehoek & Amato, "A Concise Introduction to Decentralized POMDPs", Springer 2016	doi.org/10.1007/978-3-319-28929-8
Combining local view and intent (GMU)	Arevalo et al., "Gated Multimodal Units for Information Fusion", ICLR Workshop 2017	arxiv.org/abs/1702.01992
Decoy types	MITRE D3FEND knowledge graph of countermeasures (Kaloroumakis & Smith, 2021)	d3fend.mitre.org
Capacity rule	Holz & Raynal, "Detecting Honeypots and Other Suspicious Environments", IEEE IAW 2005	doi.org/10.1109/IAW.2005.1495930
Capacity rule	Fu et al., "On Recognizing Virtual Honeypots and Countermeasures", IEEE DASC 2006	doi.org/10.1109/DASC.2006.36
Breadcrumb rule	Spitzner, "Honeytokens: The Other Honeypot", SecurityFocus 2003	No stable link (original page offline)
Strategic framing	Kiekintveld, Lisý & Píbil, "Game-Theoretic Foundations for the Strategic Use of Honeypots in Network Security", Springer 2015	doi.org/10.1007/978-3-319-14039-1_5
Closest multi-agent cyber benchmark	Kiely et al., "CAGE Challenge 4: A Scalable Multi-Agent RL Gym for Autonomous Cyber Defence", AI Magazine 2025	doi.org/10.1002/aaai.70021

3. Datasets: CAM-LDS and COMISET

	CAM-LDS (main dataset)	COMISET Lab (second dataset)
What it is	7 real attack scenarios executed in AIT's AttackBed enterprise testbed; 81 ATT&CK techniques across 13 tactics.	10 days of red-team activity logged on a single Windows host; 1.71 million labelled malicious events.
What we fixed	An earlier paper-based reconstruction dropped some attack steps. We switched to the 36 verified execution runs (1,347 labelled steps).	Our first extraction used an old 25-technique vocabulary and silently lost 43 of 53 techniques. We rebuilt it: 6,796 usable sessions.
How we used it	One attacker model per scenario, with its real entry zone, target, goal and run length. Used for training, testing and the unseen-attack test.	Benchmark for the language models alone: predicting the attacker's next technique.
What we found	Suitable: a segmented network and distinct attack scripts. All five environment checks passed.	Language models: GRU slightly beat the Transformer (72.95% vs 72.05% Top-1). Multi-agent use: failed the environment checks, because one host has no network to coordinate across. Reported as a negative result.

Sources:

CAM-LDS: Landauer et al., International Journal of Information Security, 2026 — doi.org/10.1007/s10207-026-01318-x · data: zenodo.org/records/18861762

COMISET: Data in Brief, 2025 — doi.org/10.1016/j.dib.2025.111723 · data: doi.org/10.5281/zenodo.15375145

4. Results on CAM-LDS

4.1 Attacks seen during training (10 independent runs × 500 test attacks each)

Defender team	Dwell time (steps)	Interaction depth (techniques)	Asset protection
Transformer + MAPPO (thesis model)	8.40	5.73	47%
GRU + MAPPO	8.46	5.77	47%
LSTM + MAPPO	8.45	5.74	49%
Transformer + IPPO	7.92	5.42	44%
SetEncoder + MAPPO	7.74	5.36	45%
NoHistory + MAPPO	6.58	4.70	37%
<i>Reference: no defence</i>	<i>0.00</i>	<i>0.00</i>	<i>24%</i>
<i>Reference: random decoys</i>	<i>2.10</i>	<i>1.89</i>	<i>28%</i>
<i>Reference: best fixed decoys</i>	<i>4.36</i>	<i>3.34</i>	<i>40%</i>
<i>Reference: reactive rule</i>	<i>4.58</i>	<i>3.26</i>	<i>28%</i>
<i>Reference: planner told the attack script</i>	<i>9.14</i>	<i>6.22</i>	<i>49%</i>

Dwell time = steps the attacker spends interacting with decoys. **Interaction depth** = distinct ATT&CK techniques it reveals inside decoys. **Asset protection** = share of attacks that never reach their goal.

4.2 Pre-registered hypotheses (statistically corrected for multiple tests)

Question	Result	Verdict
Does attacker-intent modelling help? (Transformer vs NoHistory)	+1.82 dwell (+28%), +1.03 depth	Supported ($p < 0.001$)
Does the order of attacker actions matter? (Transformer vs SetEncoder)	+0.66 dwell, +0.37 depth	Supported ($p \approx 0.01$)
Is the Transformer better than the GRU?	-0.06 dwell	Statistically equivalent
Is the Transformer better than the LSTM?	-0.05 dwell	Statistically equivalent
Does centralised training help? (MAPPO vs IPPO)	+0.48 dwell (+6%)	Not significant ($p = 0.06$)

Learned coordination: about one breadcrumb → partner-decoy chain per attack, almost no breadcrumbs laid without a partner, and decoy capacity rarely exceeded. The agents coordinate without sending each other messages.

4.3 Generalisation to an unseen attack script (train on 6 scripts, test on the 7th)

Defender team	Dwell on an unseen attack	Dwell on seen attacks (4.1)
Transformer + MAPPO	3.60	8.40
GRU + MAPPO	3.46	8.46
SetEncoder + MAPPO	3.56	7.74
NoHistory + MAPPO	3.69	6.58
<i>Reference: reactive rule</i>	4.62	4.58
<i>Reference: best fixed decoys</i>	3.25	4.36

Verdict: fails the pre-registered validity check. On an unseen attack every learned team loses to the simple reactive rule, and attacker-intent modelling no longer helps.

4.4 What the results mean

Genuine multi-agent learning works. The team almost doubles dwell time over the best fixed decoys and closes 84% of the gap to a planner that is told the attack script.

Knowing the attacker's intent is the main driver (+28% dwell), and the order of its actions matters too.

Attention is as good as recurrence, not better. This matches our earlier experiments.

The learned knowledge is specific to the attacks seen in training. It does not transfer to a new attack script: with only seven scripts that use largely different techniques, this is the main limitation.

5. How we will compare with published papers

No published system does the same task, so we compare capability by capability, inside our own environment.

Step	What we do	Status
1. Literature positioning	Describe what each prior system does and does not do (Table 7.1).	Done
2. Adapted baselines	Rebuild each prior system's core decision rule inside TEMARL's environment, with the same attacker, training budget, seeds and test attacks (Table 7.2).	2 of 4 done
3. Pre-registration	Write down the new comparison's hypotheses and statistical tests before running it.	Next step

How we will word the claims

We will claim: "TEMARL outperforms a placement-only (Hawkeyes-style) policy in the same environment."

We will not claim: "TEMARL outperforms Hawkeyes."

6. Why a direct comparison is not possible

Reason	What it means in practice
Different data and attackers	Other papers use IoT traffic, industrial protocols, honeypot logs or scripted attackers. TEMARL uses ATT&CK technique sequences from real CAM-LDS attack scripts.
Different simulators	Each paper builds its own network, attacker and defence rules, so a number from one simulator means something different in another.
Code or parameters not published	Huang & Zhu do not publish their simulation parameters. Our rebuild of Hawkeyes does not reproduce its published results (gaps of up to 18 points).
Different decisions	Others decide where to place a honeypot, whether to deploy deception, or how strongly to interact. TEMARL decides which decoy type each zone deploys, coordinated across four agents.
Different metrics	Others report trapping rate, engagement time or attack-path interception. TEMARL reports dwell time, interaction depth, generalisation to unseen attacks and asset protection.

Therefore
A statement like "TEMARL scores X% and Paper A scores Y%" is only fair when both run in the same environment, against the same attacker, with the same metric. Adapted baselines give us exactly that.

7. Comparison tables

7.1 What each system does

System	Learning	Multi-agent	What it decides	Uses attacker history	Main metric
Hawkeyes (Computer Networks, 2026)	Hierarchical multi-agent RL	Yes	Where to place honeypots	Partly: reacts to the observed attack stage	Defence success rate
Zambianco et al. (Computers & Security)	None: one-time optimisation	No	Which decoys to deploy, chosen once to cover ATT&CK techniques	Uses the ATT&CK catalogue, not the live attack history	Attack-path interception
Li et al. (Scientific Reports, 2025)	Single-agent RL (PPO)	No	Where to deploy deception resources	No: reconnaissance-only attacker	Defence success probability
Huang & Zhu (GameSec, 2019)	Single-agent RL (semi-Markov)	No	How strongly to engage the attacker inside each honeypot	Attacker's current position only	Engagement time, penetration risk
TEMARL (ours)	Multi-agent RL (MAPPO)	Yes, 4 agents	Which decoy type each zone deploys, every step, coordinated	Yes: learned intent from the technique history	Dwell time, interaction depth, generalisation, protection

Links: [Hawkeyes](#) · [Zambianco et al.](#) · [Li et al.](#) · [Huang & Zhu](#)

7.2 Adapted baselines inside TEMARL's environment

Baseline	Idea taken from	Capability removed	Status and result
NoHistory	RL deception without an intent model (e.g. Li et al.)	Attacker-intent modelling	Done: TEMARL +28% dwell, significant
IPPO	Independent learners (de Witt et al.)	Centralised training	Done: +6%, not significant
Placement-only	Hawkeyes' decision scope	Choice of decoy type	Planned
Static ATT&CK decoy set	Zambianco et al.	Adapting during the attack	Planned
Engagement metrics	Huang & Zhu	—	Measured for every team (dwell time, depth)

All baselines use the same environment, attack scripts, training budget (2 million steps), 10 independent runs and test attacks.